The official PHP documentation contains an example of overriding the base `SessionHandler` class to implement encryption of session data (which, as you've seen, is human readable by default). If you're interested, you can take a look at https://www.php.net/manual/en/class.sessionhandler.php

Security Considerations

Session data can potentially contain sensitive data; it's up to you (and your team) to determine the trade-off between convenience and risk to the user. As a general rule, if PII (Personally Identifiable Information, that's everything from names and addresses to medical or bank records) would be exposed by a malicious actor gaining access to a user session, then you should consider that as a high level of risk. There will undoubtedly be legislative protections in place, such as the EU's General Data Protection Regulations (GDPR), which mandate that data-controlling organizations must implement appropriate technical measures to protect users' personal data. PHP sessions are only as secure as you make them, and there are unfortunately various techniques available to gain access to a user's session.

Session Fixation

PHP's session manager is adaptive by default, which means it will accept uninitialized session IDs. This means it's possible to pass PHP the session ID you'd like it to use, and it will create that session if it doesn't already exist. If we pass a `PHPSESSID` cookie that doesn't have a session created for it, we will get one created with that ID:

```
# Remove all session files inside the cli-server container
root@0d72be74121b:/usr/src/myapp# rm /tmp/*

// index.php
session_start();

# make a curl request into the cli-server container from host
$ curl --cookie 'PHPSESSID=abc123' http://localhost:8080/index.php

# check the session files, sid abc123 now exists
root@0d72be74121b:/usr/src/myapp# rm /tmp/*
sess_abc123
```